

Coding in Rust

There are some more ways you can use the *println!* macro, as shown below:

```
fn main() {
    println!("{}", {}, {}, "Hello", "world"); // Hello, world!
    println!("{}", {}, {}, "Hello", "world"); // Hello, world!
    println!("{}", greeting, {name}!, greeting="Hello",
name="world"); // Hello, world!

    println!("{}", [1,2,3]); // [1, 2, 3]
    println!("{}", [1,2,3]);
/*
[
    1,
    2,
    3
]
*/

// format! macro is used to store the formatted STRING
let x = format!("{}", {}, {}, "Hello", "world");
println!("{}", x); // Hello, world!
}
```

Data types in Rust

A few basic data types in Rust are:

- **bool**: To define true or false values
- **char**: A single character value
- **Fixed size (bit) signed (+/-) integer types**: Denoted as *i8*, *i16*, *i32*, *i64* where the number after *i* represents the bit size
- **Fixed size (bit) unsigned (+) integer types**: Denoted as *u8*, *u16*, *u32*, *u64* where the number after *u* represents the bit size
- **Variable sized signed (+/-) integer**: Represented as *isize* in code. Covers all signed integer types
- **Variable sized unsigned (+) integer**: Represented as *usize* in code. Covers all unsigned integer types
- **32-bit floating point**: Represented as *f32* in code
- **64-bit floating point**: Represented as *f64* in code
- **Arrays**: Fixed size list of elements of the same data type. Arrays are immutable by default and even with *mut*, their element count cannot be changed
- **Tuples**: Fixed size ordered list of elements of different (or the same) data types. Tuples are also immutable by default and even with *mut*, their element count cannot be changed. Also, if you want to change an element's value, the new value should have the same data type of the previous value
- **Slice**: Used to create a reference to only part of another data structure
- **Str**: Unsized UTF-8 sequence of Unicode string slices

Examples of the above data types are shown below:

```
let a : bool = true; // bool type
let x = 'x'; // char type
let num: i16 = 12 // fixed size signed integer type
let unum: u16 = 13 // fixed size un-signed integer type
let a = [1, 2, 3]; // a[0] = 1, a[1] = 2, a[2] = 3 // array type
let a = (1, 1.5, true, 'a', "Hello, world!"); // tuple type

// example of slice
let a: [i32; 4] = [1, 2, 3, 4]; //Parent Array

let b: &[i32] = &a; //Slicing whole array
let c = &a[0..4]; // From 0th position to 4th(excluding)
let d = &a[..]; //Slicing whole array

// example of Str datatype
let a = "Hello, world."; //a: &'static str
```

Variable bindings, constants and statics in Rust

In the domain of Rust, the variables are immutable by default; hence, they are called variable bindings. To make them mutable, the *mut* keyword has been used. Rust is a statically typed language, but it doesn't need to define the data type while declaring a variable binding. The compiler is smart enough to figure out the best data type after checking the usage. The only exception to this rule is when you declare the constants and static variables. While declaring constants and statics, you must declare the type using a colon (:). Types come after the colon.

Examples for variable bindings are given below:

```
let a = true; // without specific declaration of type
let b: bool = true; // with specific declaration of type
let mut z = 5; // mutable variable binding
```

The *let* keyword is used in binding expressions. We can bind a name to a value or a function.

An example of constants is shown below:

```
const const_1: i32 = 5;
```

The *const* keyword is used to define constants.

An example of statics is given below:

```
static static_1: bool = true;
```

The *static* keyword is used to define the 'global variable' type facility.

One noteworthy difference between *static* and *const* is that the former has a fixed location in memory while *const* doesn't.